

Review for Quiz 2

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Review Topics

- Classes
 - Constructors, methods, attributes
 - `__str__()` and `__eq__()`
- State and aliasing
 - References and mutation
 - `None` and `Optional`
- Stakeholder-value matrices
 - Selecting stakeholders
 - Selecting values
 - Interactions between stakeholders and values with respect to software

How to make a class

- Class header is `class Name:` (capital letter)
- Methods: functions associated with an object
 - First parameter is `self`
- Attributes: variables shared among all methods
 - Name starts with `self.`
- Constructor: special method that is called when the object is "instantiated"
 - To initialize the attributes
 - `def __init__(self, <args>):`

`__str__()` and `__eq__()`

- Print something -> it calls `def __str__(self) -> str`
- Check if something is equal -> it calls `def __eq__(self, other: object) -> bool`
 - True if `self == other`
 - Check that `other` is the right type first
- If you don't define these methods, it uses the built-in ones

```
class TwoNumbers:
    def __init__(self, num1: int, num2: int):
        self.num1 = num1
        self.num2 = num2

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, TwoNumbers):
            return False
        return self.num1 == other.num1 and \
            self.num2 == other.num2

    def __str__(self) -> str:
        return f'({num1}, {num2})'

class TestTwoNumbers:
    def test_one_two(self) -> None:
        expected = "(2, 3)"
        assert expected == TwoNumbers(2, 3)
```

Poll: Why is this test failing?

1. We have not written the method necessary for checking whether two `TwoNumbers` are equal
2. We have not written the method necessary for converting a `TwoNumbers` into a `str`
3. The test is implicitly converting the `TwoNumbers` into a `str`
4. The test is comparing a `TwoNumbers` with a `str`

State and aliasing

- Variable that holds an object actually holds a *reference* to the object
- Can have multiple variables hold references to the same object
- Modify it using one variable -> all references to it get the modified version

None and Optional

- `None` is a value that represents the absence of a value
- `Optional[type]` is the type for a variable that might have the value `None`

Poll: What is printed?

```
class TwoNumbers:
    def __init__(self, num1: int, num2: int):
        self.num1 = num1
        self.num2 = num2

    def __str__(self) -> str:
        return f'({num1}, {num2})'

var1 = TwoNumbers(1, 2)
var2 = TwoNumbers(1, 2)
var1.num1 = 600
print(var2)
```

1. (1, 2)
2. (600, 2)

Stakeholder-Value Matrices

1. **Stakeholders:** people who are affected by the software in any way
2. **Values:** values at stake for those stakeholders when considering the software
3. **Cells in the stakeholder-value matrix:** (columns are values, rows are stakeholders), cell contains how each stakeholder's value relates to the software
4. **Analyze conflicts:** e.g., one cell says to increase x and another says to decrease x

Let's go through Practice Quiz 2 (with these related polls)

Practice quiz question: Consider the scenario where you run `git add homework3.py`, and then make a few more edits to `homework3.py` before committing. When you run `git status`, you see `homework3.py` listed under both `Changes to be committed` and `Changes not staged for commit`. Why is it listed in both sections?

Poll: Why doesn't this response get full credit?

"Only the changes made before running `git add` are included in the `Changes to be committed`"

1. Because it only answers the part about `Changes to be committed` and not `Changes not staged for commit`
2. Because it is incorrect -- the changes before `git add` are in `Changes not staged for commit`
3. Because it is correct, but unrelated to the question
4. Because it confuses the staging area with the working area

Poll: Why won't this documentation get full credit?

(Multiple free responses)

```
def multiply_substring(source: str, substring: str, multiplier: int) -> str:
    """
    Modifies the source string in place, by multiplying all substrings that match
    the substring by the multiplier.

    Parameters:
        source : str
            The string within which to multiply substrings
        substring : Optional[str]
            The substring to search and multiply
        multiplier : int
            The number of times to multiply each substring

    Returns:
        str
            The modified string with multiplied substrings
    """
```

Poll: What is this asking you to write? (What is its method signature?)

"Please add to the `ProtectedPerson` class so that if `person` is a `ProtectedPerson`, then `print(person)` prints..."

1. `def __init__(self, name: str, bodyguard: Optional[str]) -> None`
2. `def __init__(self) -> None`
3. `def __eq__(self, other: object) -> bool`
4. `def __eq__(self, other: ProtectedPerson) -> bool`
5. `def __str__(self) -> str`
6. `def __str__(self) -> object`

Poll: What is this asking you to write? (What is its method signature?)

"Please add to the **ProtectedPerson** class below so that if **person_a** and **person_b** are both **ProtectedPerson**s, then **person_a == person_b** is **True** if they have the same bodyguard name, and **False** if they have different bodyguard names (or either of them doesn't have a bodyguard)"

1. `def __init__(self, name: str, bodyguard: Optional[str]) -> None`
2. `def __init__(self) -> None`
3. `def __eq__(self, other: object) -> bool`
4. `def __eq__(self, other: ProtectedPerson) -> bool`
5. `def __str__(self) -> str`
6. `def __str__(self) -> object`

Poll: What is this asking you to write? (What is its method signature?)

"Please add to the `ProtectedPerson` class below so that when a new `ProtectedPerson` is instantiated, the client must pass a name (`str`) and a bodyguard's name (`str`) to the constructor, in that order. If the person has no bodyguard, then the bodyguard's name is `None`"

1. `def __init__(self, name: str, bodyguard: Optional[str]) -> None`
2. `def __init__(self) -> None`
3. `def __eq__(self, other: object) -> bool`
4. `def __eq__(self, other: ProtectedPerson) -> bool`
5. `def __str__(self) -> str`
6. `def __str__(self) -> object`

Using the `ProtectedPerson` class we saw before (without an overwritten `__eq__()` method), which of these will make it so `blob == glob`?

```
blob = ProtectedPerson("Blob", "Bodyguard")
```

1. `glob: ProtectedPerson = blob`
2. `glob = blob`
3. `glob = ProtectedPerson(blob.name, blob.bodyguard)`
4. `glob = ProtectedPerson("Blob", "Bodyguard")`

Poll: Which are reasons for an answer to NOT receive full credit when filling a cell in a Stakeholder-Value Matrix?

1. It isn't related to the stakeholder
2. It isn't related to the value
3. It isn't related to the software
4. It is morally questionable

Consider an online shopping website where people can order clothing online to be shipped to them. Please fill in the cells in the bottom row of the following Stakeholder-value matrix, where the stakeholder is the shipping worker.

Poll: What can you say for the "Shipping distance" cell?

Consider an online shopping website where people can order clothing online to be shipped to them. Please fill in the cells in the bottom row of the following Stakeholder-value matrix, where the stakeholder is the shipping worker.

Poll: What can you say for the "Physical safety" cell?

Poll:

- 1. What is your main takeaway from today?**
- 2. What would you like to revisit next time?**